

作业1

24371277 刘哲晗

问题1

多道程序设计

指在内存中的多个相互独立的程序，使他们并发的处于运行状态，共享内存资源，轮流占有CPU，交替执行

与分时系统的区别

目标上：多道程序系统是为了提高系统资源利用率，吞吐量；而分时系统是为了提供快速响应时间，支持多用户交互

交互性：前者多为批处理，后者强调交互性，用户实时输入输出

响应时间：后者要求短时间响应

问题2

什么原因推动了操作系统从批处理发展到多道程序，进而发展到分时系统？

硬件成本与使用效率的矛盾推动了发展：

1. **批处理** → **多道程序**：CPU与I/O设备速度差异巨大，单道程序运行时CPU大量时间闲置等待I/O。为提高CPU利用率和系统吞吐量，让多个程序同时驻留内存，当一个程序等待I/O时CPU可切换运行另一个程序。
2. **多道程序** → **分时系统**：多道程序系统以批处理为主，缺乏交互性。随着终端设备出现和多用户交互需求增长，需要系统能快速响应用户输入。分时系统通过时间片轮转机制，让每个用户都能获得及时响应，实现人机交互。

问题3

什么是陷阱？与中断的区别是什么？什么是系统调用？

陷阱 (Trap)：程序有意执行特殊指令触发的同步事件，通常用于程序主动请求操作系统服务。

与中断的区别：

维度	陷阱	中断
触发原因	程序有意执行特殊指令	外部设备或时钟发出信号
同步性	同步	异步
返回行为	通常返回下一条指令	通常返回下一条指令
例子	系统调用	定时器中断、I/O完成中断

系统调用：一种受控陷入内核的方式。用户程序通过特定指令进入内核态，CPU切换到内核栈，根据中断向量执行内核处理程序，处理结束后返回用户态。本质是同步陷阱。

问题4

判断：可移植的操作系统可以从一个系统架构移植到另外一个系统架构而无需修改。

(1) 为什么构建完全可移植的OS是不可能的？

操作系统直接与硬件交互，而不同架构的硬件存在本质差异：

- 不同CPU有不同指令集架构（ISA），如x86、ARM、RISC-V指令编码和寄存器组织完全不同
- 中断处理机制、内存管理单元（MMU）、页表格式等硬件机制架构差异巨大
- 启动流程、设备寄存器、I/O操作方式等硬件相关代码必须针对特定架构编写
- 即使采用分层设计，最底层的硬件抽象层（HAL）也必须为每个架构单独实现

(2) 设计高度可移植的OS需要设计的两个层次：

1. **硬件抽象层（HAL）/架构相关层：**封装所有与具体CPU架构、硬件平台相关的代码，包括启动代码、中断处理、上下文切换、MMU操作、设备驱动等。这一层需要为每个支持的架构单独实现。
2. **架构无关层/核心服务层：**操作系统的核心逻辑，包括进程调度算法、文件系统、内存管理逻辑、系统调用处理、IPC机制等。这一层代码完全不依赖具体硬件，通过调用HAL提供的统一接口工作，移植时无需修改。

问题5

一对相互矛盾的设计实例：

响应时间 vs 吞吐量

- **响应时间：**系统对请求做出反应的时间，追求越快越好
- **吞吐量：**单位时间内系统处理的请求数量，追求越大越好

矛盾所在：为了提高吞吐量，操作系统通常会采用批处理方式，将多个请求累积起来一次性处理，减少上下文切换开销，提高资源利用率。但这样会增加每个请求的等待时间，导致响应时间变长。

反之，为了保证快速响应时间，系统需要频繁地在不同任务间切换，及时处理每个请求。但上下文切换本身会消耗CPU时间，反而降低了系统整体吞吐量。

例如：时间片设置过长→响应慢但吞吐高；时间片设置过短→响应快但吞吐低。

问题6

最简单的启动需要哪些功能：

1. **硬件初始化：**CPU、内存控制器、基本寄存器的初始化，设置好运行环境
2. **内核镜像加载：**将操作系统内核从存储介质加载到内存中
3. **栈设置：**初始化内核栈，为C代码运行准备环境
4. **跳转到内核入口：**从汇编启动代码跳转到C语言编写的内核主函数
5. **核心子系统初始化：**内存管理、进程调度、中断向量表等核心机制初始化
6. **启动第一个用户进程：**创建并运行第一个用户态程序（如init进程或shell）